# LLM-Driven Iterative Software Testcase Generation Tool

## 1. Project Goals and Research Questions

This project revolves around an LLM-driven automated test generation tool. The `improved agent` is the primary research subject of this paper, while the `naive agent` serves as a control baseline for comparative reference.
Specifically, the `naive agent` represents a conservative single-round generation approach, used to answer a fundamental question: what baseline level can test generation achieve without stronger prompt design and Coverage-guided iteration?

Building upon this, the real focus of this project is:
**Whether the `improved agent` can generate higher-quality test cases on real-world open-source projects through stronger prompt design and white-box iteration mechanisms.**

## 2. System Design, Input, and Output

This section primarily introduces the method core, the `improved agent`. The `naive agent` is only a control baseline and is not the focus of the system design.

### 2.1 Overall Process

The overall system workflow is as follows:

1. Read the target function's requirement and/or source code.
2. Construct a prompt based on the mode and request the LLM to generate structured test cases.
3. Normalize the output into a unified JSON format, including `obligations` and `test_groups`.
4. Execute the tests against the real implementation to determine Correctness.
5. Collect Code Coverage, obtaining metrics for `Stmt / Branch / Func / Line`.
6. Align with manual golden tests for semantic category matching to obtain the `Golden Tests Match Ratio`.
7. If in the `improved` white-box mode, continue using coverage feedback for patch iteration.

### 2.2 Input

The system inputs primarily include the `requirement` and the `project code base`.
The project supports three modes:

- `blackbox`: Provides only the requirement.
- `whitebox`: Provides both the requirement and the source code.
- `whitebox_code_only`: Provides only the source code.

For `improved` white-box iteration, each round additionally inputs:

- The current complete suite;
- `coverage_details`;

- `uncovered_details`.

## 2.3 Output

The system output is in strict JSON format as `test cases`.
The unified format is as follows:

- Top-level keys:
  - `obligations`
  - `test_groups`
- Each `test_group` contains:
  - `title`
  - `validator`
  - `args`
  - `valid`
  - `invalid`
  - `rationale`
  - `obligations`

## 2.4 `naive agent`: Control Baseline

The design goal of the `naive agent` is to provide a conservative first-round control baseline.

Its characteristics are:

- Generates only the first "small and stable" version of the suite;
- Does not actively pursue completeness;
- Does not perform deep branch hunting;
- Favors common cases and obvious boundaries;
- Even in white-box mode, it only treats code as auxiliary information to "discover obvious branches/guards."

Therefore, the `naive agent` can be summarized as:

**A conservative, low-risk, Correctness-oriented baseline generation scheme.**

## 2.5 `improved agent`: Main Method Subject

The core design of the `improved agent` has two focal points:

1. **Prompt Enhancement**
2. **White-box Iteration**

### 2.5.1 `blackbox` Mode

In `blackbox` mode, the `improved agent` doesn't just "write a few cases from the requirement" but explicitly requests the model to:

- Perform equivalence partitioning;

- Perform boundary value analysis;
- Perform option-combination testing;
- First propose obligations, then cluster categories, and finally expand test groups;
- Conduct a self-audit to check if rare options, boundaries, and option flips were missed.

In contrast, the `naive blackbox` only performs conservative single-round baseline generation. Therefore, the advantage of `improved blackbox` primarily comes from **stronger structured prompt design**, rather than iteration.

### 2.5.2 `whitebox` Mode

The `improved agent` in `whitebox` mode is the most central research object of the entire project.

Its design consists of two layers:

1. **Static Prompt Enhancement**
    - Explicitly requires statement coverage, branch coverage, and condition-oriented reasoning;
    - Simultaneously focuses on requirement-visible behavior and code-visible structure;
    - Guides the model to enumerate structural white-box obligations;
    - Guides focus toward early returns, helper-sensitive paths, and boundary-triggered branches.
2. **Dynamic Coverage Iteration**
    - Each round provides the current complete suite;
    - Provides the `requirement`, `coverage_details`, and `uncovered_details`;
    - The model outputs only patch groups;
    - The system combines `current suite + patch` to obtain the next round's suite.

Thus, the value of `improved whitebox` is not just "having seen the code," but:

**First generating a structured initial version through stronger white-box prompts, then incrementally filling paths through coverage-guided patch iteration.**

### 2.5.3 `whitebox_code_only` Mode

In `whitebox_code_only` mode, the system no longer provides requirements, only the source code.

The `improved agent` will:

- Explicitly define the task as code-only white-box testing;
- Infer externally visible behavior conservatively only from code-visible contracts;
- Emphasize avoiding speculative claims lacking code support;
- Use the current suite, coverage details, and uncovered details during iteration.

The research significance of this mode is:

- To test "whether LLMs can make decisions close to white-box testing when only code is available";
- To expose the risks of semantic drift when requirements are missing.

## 2.6 The advantage of `improved` primarily comes from prompt + iteration

The performance improvement of `improved` should not be simply understood as "a stronger model."
More accurately, it comes from two types of engineering designs:

1. **Prompt-level Enhancements**
    - Improving semantic category expansion in blackbox;
    - Improving path and structural awareness in whitebox;
    - Restricting unsupported inferences in code-only.
2. **Iteration-level Enhancements**
    - Utilizing coverage / uncovered details in white-box scenarios for patch refinement to improve Coverage metrics and Golden Tests Match Ratio;
    - Retaining existing effective groups via `suite + patch`, avoiding the disruption caused by full-suite rewriting.

Thus, the `improved agent` constitutes the primary method design of this paper, while the `naive agent` provides a stable control baseline.

# 3. Reasons for Choosing `validator.js` as the Test Library

`validator.js` was chosen as the experimental subject for the following reasons:

1. It is a real, mature, and widely used open-source project, not an artificially constructed toy benchmark.
2. It simultaneously possesses requirements, source code, and human-expert-written test sets, making it suitable for joint evaluation of black-box and white-box testing.
3. Its function interfaces exhibit diversity, ranging from relatively simple validators to those with option parameters, boundary conditions, and complex branches.

Importantly, `validator.js` provides two types of ground truth:

- **The Real Implementation**: Used to judge whether the generated tests execute correctly;
- **Manual Golden Test Set**: Used to evaluate the proximity of generated tests to expert tests from the perspective of equivalence classes/semantic categories.

Thus, it is highly suitable for evaluating:

- Correctness;
- Coverage;
- Golden Tests Match Ratio.

# 4. Three Core Evaluation Metrics

## 4.1 Correctness

Meaning: The proportion of generated tests that judge correctly after execution on the real implementation.

It reflects:

- Whether the tests can actually execute;
- Whether the test assertions are correct;
- Whether the LLM correctly understands the external behavior of the function.

Reason for selection:
If test assertions are wrong, even high Coverage cannot be considered high-quality testing.

## 4.2 Coverage

Includes:

- `Stmt Cov`
- `Branch Cov`
- `Func Cov`
- `Line Cov`

It reflects:

- Whether the tests have reached more implementation paths;
- Whether white-box information effectively translates into path exploration capability;
- Whether iterative patches target previously uncovered code locations.

## 4.3 Golden Tests Match Ratio

Meaning: The overlap ratio in semantic categories between generated tests and manual golden tests.

It reflects:

- Whether major equivalence classes are covered;
- Whether the semantic structure of the tests is close to the expert's design;
- Whether the generated tests are merely "accidentally correct" or also possess reasonable test design.

## 4.4 Why all three metrics must be viewed simultaneously

One of the most important findings of this project is:

**Correctness, Coverage, and Golden Tests Match Ratio are related but not equivalent.**

For example, on `isCurrency`, the following often occurs:

- Coverage is high, even reaching `100%`;
- Golden Tests Match Ratio is also high;

- However, Correctness remains significantly low.

This indicates:

- Reaching a code path doesn't mean the assertion is correct;
- Similarity to expert test categories doesn't mean each case is correct;
- Therefore, experiments cannot only look at Coverage and Golden Tests Match Ratio; Correctness is also an indispensable metric.

On other functions, it also frequently occurs that:

- Correctness is high, even reaching `100%`;
- Golden Tests Match Ratio and Coverage are significantly lower.

This indicates:

- The agent is overly conservative in generated test cases, sacrificing creativity for Correctness aligned with the requirements.
- This also demonstrates the necessity of the Coverage and Golden Tests Match Ratio metrics.

# 5. Experimental Design

The final set of functions used for conclusions is:

- `isEmail`
- `isURL`
- `isFQDN`
- `isCurrency`
- `isCreditCard`

The total number of experiments is:

- `5` functions
- `3` modes
- `2` types of agents

Totaling `30` experiments.

Specifically:

- `naive` serves as the baseline;
- `improved` is the primary method;
- Comparisons are made across three modes:
  - `blackbox`
  - `whitebox`
  - `whitebox_code_only`

Functions that were "too simple" or had "unstable golden extraction" were removed because practice showed:

- Too simple functions make all methods perform well, diluting differences on truly difficult functions;

- High-quality benchmarks need "differentiation + sufficient golden tests."

# 6. Experimental Results

## 6.1 Overall Results

The final experimental results come from the complete data in `experiment_final_1.md`:

- Total experiments: `30`
- Successful experiments: `30`
- Total LLM requests: `140`
- Total token consumption: `915,473`

## 6.2 Summary by `mode + approach`

| Agent | Mode | Avg Tokens | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Func Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|---|---|---|
| `naive` | `blackbox` | 11259.80 | 0.8826 | 81.78 | 74.89 | 100.00 | 81.72 | 0.5942 |
| `naive` | `whitebox` | 19494.40 | 0.9167 | 84.27 | 78.74 | 100.00 | 84.33 | 0.7714 |
| `naive` | `whitebox_code_only` | 18603.80 | 0.8855 | 83.71 | 75.11 | 100.00 | 84.49 | 0.5002 |
| `improved` | `blackbox` | 21537.20 | 0.8823 | 83.98 | 78.53 | 100.00 | 84.09 | 0.9667 |
| `improved` | `whitebox` | 58264.80 | 0.8132 | 95.19 | 92.31 | 100.00 | 95.02 | 0.9857 |
| `improved` | `whitebox_code_only` | 53934.60 | 0.7793 | 90.19 | 81.70 | 100.00 | 89.66 | 0.9312 |

## 6.3 Complete Results Matrix

| Validator | Agent | Mode | Status | Correct/Total | Correctness Rate | Stmt Cov | Branch Cov | Func Cov | Line Cov | Golden Tests Match Ratio | Total Tokens | Note |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| isEmail | naive | blackbox | success | 44/46 | 0.9565 | 73.81 | 68.42 | 100.00 | 73.75 | 0.5000 | 12093 | ok |
| isEmail | naive | whitebox | success | 41/42 | 0.9762 | 73.81 | 68.42 | 100.00 | 73.75 | 0.6429 | 13194 | ok |
| isEmail | naive | whitebox_code_only | success | 43/47 | 0.9149 | 66.67 | 52.63 | 100.00 | 67.50 | 0.6429 | 13120 | ok |
| isEmail | improved | blackbox | success | 40/43 | 0.9302 | 79.76 | 75.00 | 100.00 | 80.00 | 1.0000 | 14843 | ok |
| isEmail | improved | whitebox | success | 47/57 | 0.8246 | 94.05 | 90.79 | 100.00 | 93.75 | 0.9286 | 98631 | ok |
| isEmail | improved | whitebox_code_only | success | 36/47 | 0.7660 | 97.62 | 92.11 | 100.00 | 97.50 | 0.9286 | 50120 | ok |
| isURL | naive | blackbox | success | 44/50 | 0.8800 | 64.41 | 52.10 | 100.00 | 65.79 | 0.8182 | 14416 | ok |
| isURL | naive | whitebox | success | 52/64 | 0.8125 | 66.10 | 60.50 | 100.00 | 67.54 | 0.8182 | 18662 | ok |
| isURL | naive | whitebox_code_only | success | 35/41 | 0.8537 | 72.88 | 63.87 | 100.00 | 74.56 | 0.5455 | 15721 | ok |
| isURL | improved | blackbox | success | 34/41 | 0.8293 | 64.41 | 54.62 | 100.00 | 65.79 | 1.0000 | 55039 | ok |
| isURL | improved | whitebox | success | 54/64 | 0.8438 | 96.61 | 86.55 | 100.00 | 96.49 | 1.0000 | 76002 | ok |
| isURL | improved | whitebox_code_only | success | 36/39 | 0.9231 | 78.81 | 66.39 | 100.00 | 78.07 | 0.7273 | 146922 | ok |
| isFQDN | naive | blackbox | success | 43/48 | 0.8958 | 85.29 | 84.21 | 100.00 | 84.85 | 0.8333 | 7897 | ok |
| isFQDN | naive | whitebox | success | 59/60 | 0.9833 | 91.18 | 89.47 | 100.00 | 90.91 | 0.8333 | 40602 | ok |
| isFQDN | naive | whitebox_code_only | success | 13/13 | 1.0000 | 91.18 | 92.11 | 100.00 | 90.91 | 0.1667 | 36714 | ok |
| isFQDN | improved | blackbox | success | 28/30 | 0.9333 | 82.35 | 81.58 | 100.00 | 81.82 | 0.8333 | 8538 | ok |
| isFQDN | improved | whitebox | success | 27/35 | 0.7714 | 85.29 | 84.21 | 100.00 | 84.85 | 1.0000 | 65143 | ok |
| isFQDN | improved | whitebox_code_only | success | 22/26 | 0.8462 | 91.18 | 92.11 | 100.00 | 90.91 | 1.0000 | 26464 | ok |
| isCurrency | naive | blackbox | success | 48/52 | 0.9231 | 85.37 | 75.00 | 100.00 | 84.21 | 0.3750 | 13756 | ok |
| isCurrency | naive | whitebox | success | 74/82 | 0.9024 | 90.24 | 80.56 | 100.00 | 89.47 | 0.5625 | 16352 | ok |
| isCurrency | naive | whitebox_code_only | success | 72/96 | 0.7500 | 87.80 | 72.22 | 100.00 | 89.47 | 0.8125 | 20465 | ok |

| Validator | Agent | Mode | Status | Correct/Total | Correctness Rate | Stmt Cov | Branch Cov | Func Cov | Line Cov | Golden Tests Match Ratio | Total Tokens | Note |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| isCurrency | improved | blackbox | success | 52/62 | 0.8387 | 97.56 | 97.22 | 100.00 | 97.37 | 1.0000 | 20067 | ok |
| isCurrency | improved | whitebox | success | 45/67 | 0.6716 | 100.00 | 100.00 | 100.00 | 100.00 | 1.0000 | 39595 | ok |
| isCurrency | improved | whitebox_code_only | success | 35/49 | 0.7143 | 100.00 | 100.00 | 100.00 | 100.00 | 1.0000 | 23228 | ok |
| isCreditCard | naive | blackbox | success | 25/33 | 0.7576 | 100.00 | 94.74 | 100.00 | 100.00 | 0.4444 | 8137 | ok |
| isCreditCard | naive | whitebox | success | 20/22 | 0.9091 | 100.00 | 94.74 | 100.00 | 100.00 | 1.0000 | 8662 | ok |
| isCreditCard | naive | whitebox_code_only | success | 10/11 | 0.9091 | 100.00 | 94.74 | 100.00 | 100.00 | 0.3333 | 6999 | ok |
| isCreditCard | improved | blackbox | success | 22/25 | 0.8800 | 95.83 | 84.21 | 100.00 | 95.45 | 1.0000 | 9199 | ok |
| isCreditCard | improved | whitebox | success | 21/22 | 0.9545 | 100.00 | 100.00 | 100.00 | 100.00 | 1.0000 | 11953 | ok |
| isCreditCard | improved | whitebox_code_only | success | 11/17 | 0.6471 | 83.33 | 57.89 | 100.00 | 81.82 | 1.0000 | 22939 | ok |

# 7. Detailed Comparative Analysis

This section analyzes the results from two dimensions:

1. Within the same mode, what `improved` gained relative to the baseline `naive`;
2. Within the same agent internal, what information gain or risk different modes brought.

## 7.1 Same Mode: `naive` baseline vs `improved`

### 7.1.1 `blackbox`

| Agent | Avg Tokens | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|---|
| `naive blackbox` | 11259.80 | 0.8826 | 81.78 | 74.89 | 81.72 | 0.5942 |
| `improved blackbox` | 21537.20 | 0.8823 | 83.98 | 78.53 | 84.09 | 0.9667 |

Conclusion:

- `Avg Correctness` (Correctness) is nearly identical;
- The `Avg Golden Tests Match Ratio` of `improved` increased significantly (from 0.5942 to 0.9667), and various Code Coverage metrics (`Avg Stmt/Branch/Line Cov`) also saw slight increases;
- Costs (Tokens consumption) approximately doubled.

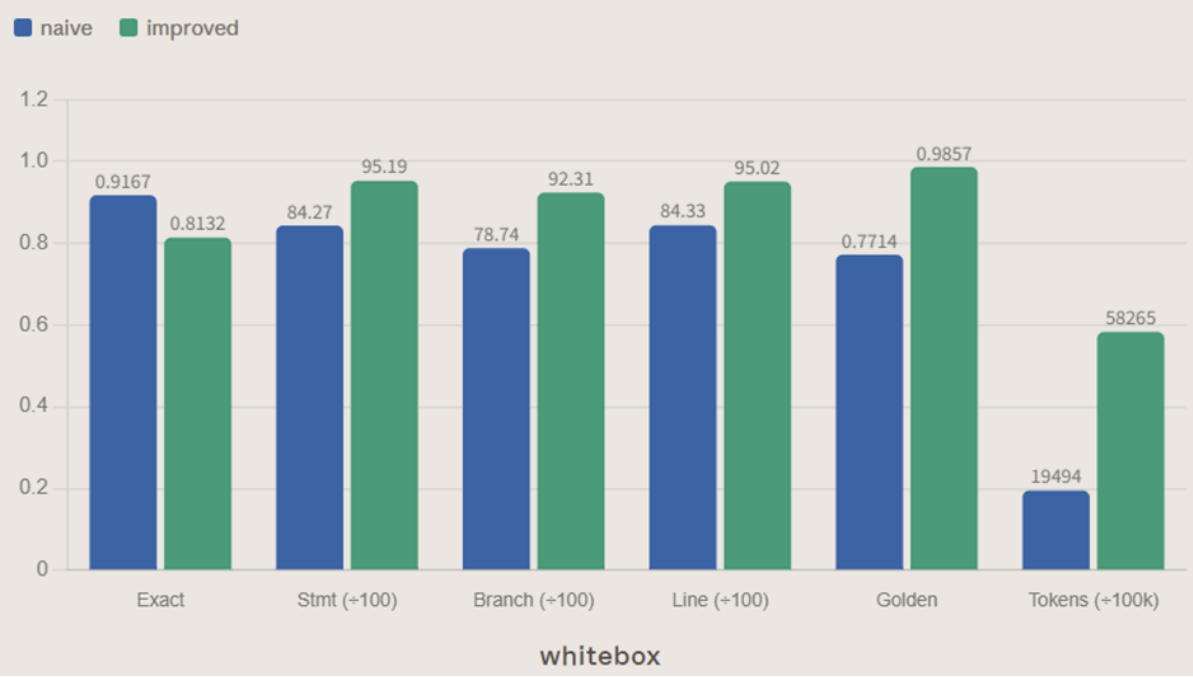
Reasons:

- `naive blackbox` is just a conservative baseline;
- `improved blackbox` uses obligations extraction, category clustering, and self-audit;
- Therefore, it more easily covers semantic categories and option families scattered across requirements, as directly reflected in the significantly boosted `Avg Golden Tests Match Ratio`.

This indicates that in the blackbox scenario, the primary gain of `improved` first comes from **prompt enhancement**, not iteration.

### 7.1.2 `whitebox`

| Agent | Avg Tokens | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|---|
| `naive whitebox` | 19494.40 | 0.9167 | 84.27 | 78.74 | 84.33 | 0.7714 |
| `improved whitebox` | 58264.80 | 0.8132 | 95.19 | 92.31 | 95.02 | 0.9857 |




Conclusion:

- The baseline `naive` has stronger `Avg Correctness` (0.9167 vs 0.8132);
- The various Code Coverage metrics and `Avg Golden Tests Match Ratio` of `improved` are significantly leading (e.g., `Avg Stmt / Line Cov` reached 95%);
- Costs for `improved` are significantly higher.

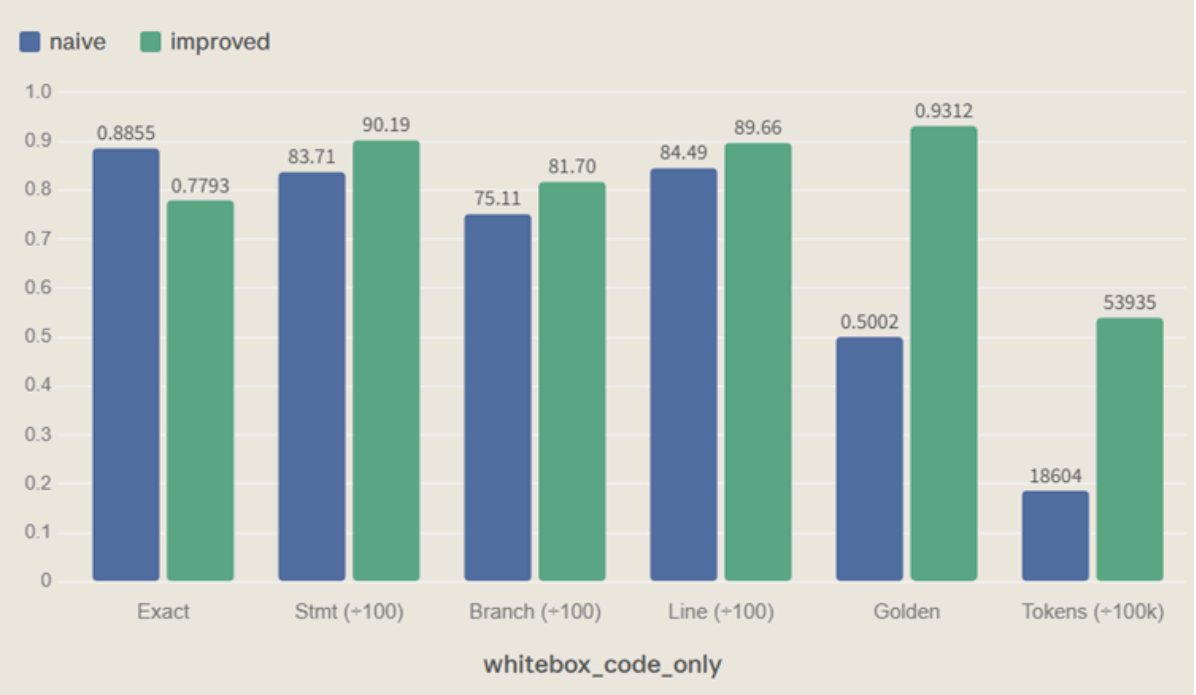
Reasons:

- `naive` is more conservative and does not actively chase coverage maximality, making it easier to maintain a high `Avg Correctness`;
- `improved whitebox` combines stronger white-box prompts with coverage-guided patch iteration;
- Thus, it is more likely to capture branches and semantic categories but also more probe to errors in parameters, assertions, and boundary details.

This shows:
The primary advantage of `improved whitebox` is not Correctness, but outstanding **structural coverage (Coverage) and semantic coverage (Golden Tests Match Ratio)**.

### 7.1.3 `whitebox_code_only`

| Agent | Avg Tokens | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|---|
| `naive whitebox_code_only` | 18603.80 | 0.8855 | 83.71 | 75.11 | 84.49 | 0.5002 |
| `improved whitebox_code_only` | 53934.60 | 0.7793 | 90.19 | 81.70 | 89.66 | 0.9312 |




Conclusion:

- The `Avg Golden Tests Match Ratio` and various Code Coverage metrics of `improved` comprehensively exceed the baseline;
- However, the baseline `naive` retains more stable `Avg Correctness`;
- `improved` cost is high.

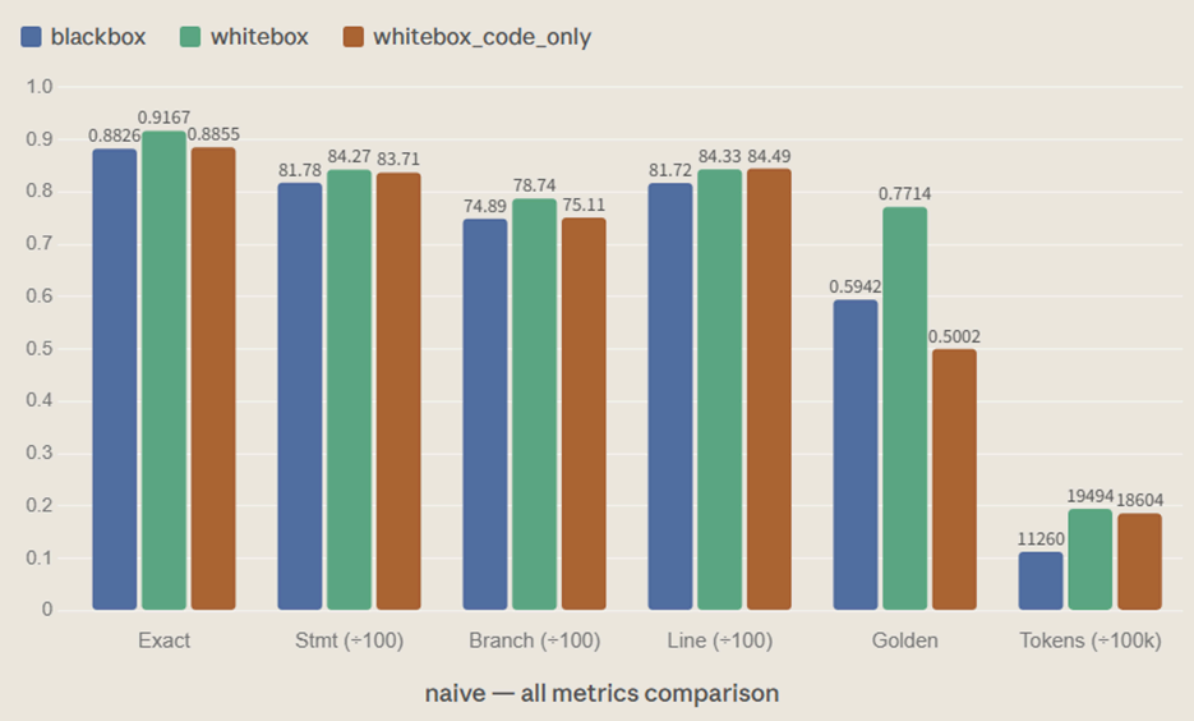
Reasons:

- In the code-only scenario, there are no requirement constraints;
- `improved` more actively expands test categories based on code branches, raising metrics like `Avg Stmt/Branch/Line Cov`;
- But more aggressive inference of external behavior from code makes it more susceptible to "looking correct at the path level but being semantically incorrect," leading to a significant drop in Correctness.

## 7.2 Within Same Agent: Comparison of Modes

### 7.2.1 `naive` baseline Internal Comparison

| Mode | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|
| `blackbox` | 0.8826 | 81.78 | 74.89 | 81.72 | 0.5942 |
| `whitebox` | 0.9167 | 84.27 | 78.74 | 84.33 | 0.7714 |
| `whitebox_code_only` | 0.8855 | 83.71 | 75.11 | 84.49 | 0.5002 |




Observations:

1. `naive whitebox` is undoubtedly the best-performing mode in the baseline, leading across all metrics.

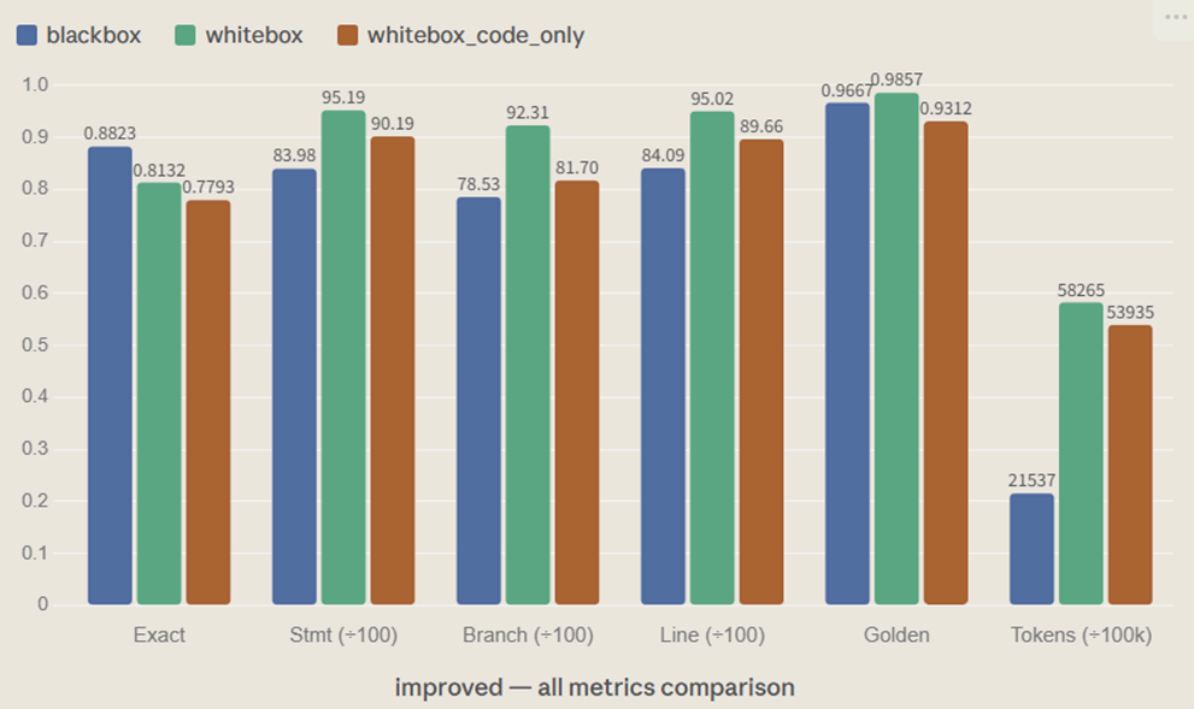
2. Although `naive blackbox` has lower coverage (e.g., `Avg Stmt Cov` 81.78) and `Avg Golden Tests Match Ratio` than `whitebox`, it has the lowest cost and maintains a high `Avg Correctness`.
3. `naive whitebox_code_only` still maintains decent `Avg Correctness` and stable coverage but yields the lowest `Avg Golden Tests Match Ratio` (0.5002).

Explanation:

- For the baseline, the combination of requirement + code provides the most balanced information;
- With only the requirement, the model can still write a stable first-round baseline;
- With only code, the lack of requirement constraints leads the model to only catch obvious branches to maintain coverage, but easily misses the semantic structure present in expert tests.

### 7.2.2 `improved` Internal Comparison

| Mode | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|
| `blackbox` | 0.8823 | 83.98 | 78.53 | 84.09 | 0.9667 |
| `whitebox` | 0.8132 | 95.19 | 92.31 | 95.02 | 0.9857 |
| `whitebox_code_only` | 0.7793 | 90.19 | 81.70 | 89.66 | 0.9312 |




Observations:

1. `improved blackbox` is the most balanced mode. Its Correctness (`Avg Correctness`) is the highest among the three modes, and it achieves an equally extremely high `Avg Golden Tests Match Ratio`.

2. `improved whitebox` is the strongest coverage mode. It achieves the best Code Coverage (multiple metrics breaking 90%) and nearly perfect `Avg Golden Tests Match Ratio`, at the cost of sacrificing some Correctness.
3. `improved whitebox_code_only` carries the highest risk. Due to over-chasing coverage paths, its `Avg Correctness` drops most sharply, although it still maintains impressive coverage and match ratios even without requirements.

Explanation:

- `improved blackbox` primarily relies on prompt enhancement, which significantly improves semantic category generalization and stays away from path pitfalls, making it the most balanced;
- `improved whitebox` combines code logic with uncovered feedback, maximizing the exploration domain of coverage paths;
- `improved whitebox_code_only` also strives to fill unknown paths, but the absence of requirements amplifies the security risk of semantic drift.

# 8. Ablation Experiments

This section focus on answering two questions:

1. Should requirement re-checking be added to `improved whitebox`?
2. Why is `suite + patch` better than pure suite rewriting?

For a fair comparison, this section does not directly compare total tables across rounds but follows the same fixed `5` function scope based on `experiment_ablation_1.md`, `experiment_ablation_2.md`, and `experiment_final_1.md`:

- `isEmail`
- `isURL`
- `isFQDN`
- `isCurrency`
- `isCreditCard`

Then, only compare the averages of `improved whitebox`.

## 8.1 Ablation of Re-checking Mechanism: Whether to Add Requirement Re-checking

`experiment_ablation_1.md` corresponds to the version of `improved whitebox` with "requirement re-checking / pruning," while the final version removed this mechanism.

Under the same `5`-function scope, the comparison for `improved whitebox` is as follows:

| Version | Avg Tokens | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Func Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|---|---|
| `experiment_ablation_1` (with requirement re-check) | 79361.20 | 0.8849 | 88.38 | 82.59 | 100.00 | 88.49 | 0.7776 |
| `final` (final version) | 58264.80 | 0.8132 | 95.19 | 92.31 | 100.00 | 95.02 | 0.9857 |

This result shows:

1. Requirement re-checking did indeed maintain higher `Avg Correctness` in this dataset;
2. But the costs were:
    - Tokens were higher;
    - Coverage (Coverage) was lower;
    - Golden Tests Match Ratio (`Avg Golden Tests Match Ratio`) was significantly lower.

Therefore, for `improved whitebox`, the re-check mechanism did not make it a "more balanced" method but rather weakened its most valuable abilities:

- Path exploration;
- Semantic category expansion.

Final project judgment:
**Requirement re-checking is the wrong direction.**

## 8.2 Ablation of Iteration Method: Why `suite + patch` is better than full suite rewriting

`experiment_ablation_2.md` corresponds to the version after removing the re-check mechanism; the final version further converged to the current `suite + patch` white-box iteration method.

Under the same `5`-function scope, the comparison for `improved whitebox` is as follows:

| Version | Avg Tokens | Avg Correctness | Avg Stmt Cov | Avg Branch Cov | Avg Func Cov | Avg Line Cov | Avg Golden Tests Match Ratio |
|---|---|---|---|---|---|---|---|
| `experiment_ablation_2` (full suite iteration) | 68342.80 | 0.8164 | 88.74 | 81.49 | 100.00 | 89.37 | 0.8430 |
| `final` (suite + patch) | 58264.80 | 0.8132 | 95.19 | 92.31 | 100.00 | 95.02 | 0.9857 |

This result shows:

1. `suite + patch` significantly reduced Token costs;
2. `suite + patch` clearly improved multiple Coverage metrics;
3. `suite + patch` clearly improved `Avg Golden Tests Match Ratio`;
4. `Avg Correctness` remained roughly the same, dropping only slightly (0.8164 vs 0.8132).

Therefore, the more accurate conclusion is not " `suite + patch` significantly improved Correctness," but:

**While almost not damaging `Avg Correctness`, `suite + patch` significantly improved Code Coverage and Golden Tests Match Ratio (`Avg Golden Tests Match Ratio`), while reducing costs.**

# 9. Project Practical Experience

This section does not repeat the ablation experiments but summarizes the more general experience gained during the entire R&D and experimental process.

## 9.1 Simple Functions Must Be Excluded

Functions that are too simple make all methods perform very well, diluting the real differences in difficult functions.
Therefore, benchmark design itself is a key component of experimental credibility.

## 9.2 High Coverage Does Not Equal High-Quality Testing

In cases like `isCurrency`, multiple instances occurred where:

- Coverage metrics (e.g., `Avg Stmt Cov`) were extremely high;
- `Avg Golden Tests Match Ratio` (matching expert test categories) was extremely high;
- But Correctness (`Avg Correctness`) was low.

This shows that when AI generates tests, it cannot only chase coverage.
Without Correctness constraints, high Coverage may just mean "reaching many paths" without actually writing correct test cases.

## 9.3 Code-only mode is inherently more dangerous

When only code is given, the model more easily learns implementation structures but more easily loses the constraints requirements impose on input semantics, boundaries, and external behavior.
Therefore, `whitebox_code_only` is better suited as a supplementary perspective rather than the default main mode.

## 9.4 Baselines Are Extremely Important

Without a `naive baseline`, many complex designs might look "very advanced."
But once compared with a stable, conservative, low-risk baseline, it becomes apparent that:

- Some complex designs only increased token costs;
- Some complex designs increased coverage but did not increase Correctness;
- Only a few designs are truly worth keeping.

Therefore, the baseline in this project is not a trivial side character but a key reference for verifying if R&D improvements are truly effective.

# 10. Comparison with Human Expert Repairs

To fully evaluate the positioning of this method, it's also necessary to compare it with traditional non-AI methods.
This project did not implement a full automatic traditional test generator, but it clearly compares the strengths and weaknesses of the AI method versus manual/expert repair methods.

## 10.1 Advantages of Human Expert Repair

- Correctness (Exact Match) is more stable;
- Fewer requirement misunderstandings;
- Easier to confirm key boundary conditions;
- Better suited as the final gatekeeper for high-risk cases.

## 10.2 Advantages of the AI Method

- Fast generation speed;
- Able to rapidly expand candidate tests and semantic categories;
- Stronger path exploration capability in white-box scenarios;
- Very suitable for initial test drafts and Coverage exploration.

## 10.3 Deficiencies of the AI Method

- Misinterprets requirements;
- Writes incorrect parameter structures or assertions;
- High Coverage does not mean high Correctness;
- More prone to semantic drift when requirements are missing.

## 10.4 A more realistic positioning

Therefore, this project believes AI is better suited as:

**A test design accelerator / draft generator / Coverage exploration tool**

rather than a complete replacement for human test engineers.

A more realistic workflow is:

1. First have an AI generate candidate test sets;
2. Then have a human or rule-based check perform filtering, correction, and final confirmation.

# 11. Generalization Ability of the Method

This project focuses on generalization, rather than just looking at whether a single function "happened to succeed."

## 11.1 Generalization Across Function Types

The final `5` functions included do not belong to the same simple pattern:

- `isEmail`: complex rules, many options;
- `isURL`: complex path and option combinations;
- `isFQDN`: clear structure but many boundaries;
- `isCurrency`: obvious coupling of format and options;
- `isCreditCard`: strong validation rules, sensitive boundaries.

Despite different function styles, stable patterns were repeatedly observed in the experiments:

- baseline `naive` is more robust across various Correctness metrics;
- `improved` obtained very strong Code Coverage and golden matching abilities;
- code-only carries high risk easily leading to semantic drift;
- `suite + patch` is superior to full suite rewriting.

This indicates the findings do not just hold on one function but are repeatedly observable across multiple different validators.

## 11.2 Generalization Across Input Modes

This project covers:

- Providing only the requirement;
- Requirement + code;
- Providing only the code.

Thus, the method's generalization is reflected not only "cross-function" but also "cross-information conditions."

Experiments show:

- When requirements are available, the model more easily maintains and improves its test Correctness;
- When code is available, the model more easily targets improvements in the breadth of Code Coverage;
- When only code is available, the model is more likely to experience semantic drift.

This indicates that the conclusions have explanatory power for different test input scenarios.

## 11.3 Generalization Across Design Versions

This project went through multiple rounds of design evolution:

- With requirement re-checking + `suite + patch` iteration;
- Without requirement re-checking + full suite rewriting;
- Without requirement re-checking + `suite + patch` iteration.

In these design version variations, some phenomena remained consistently stable:

- The core focus and advantage of `improved whitebox` are outstanding structural coverage `Coverage` and `Avg Golden Tests Match Ratio`, rather than superior Correctness;
- The core advantage of baseline `naive` lies in its long-term stability of Correctness;
- Only-code mode is more volatile and untrustworthy in the absence of supervision.

This indicates the project's conclusions did not come accidentally from one experiment but possess some cross-version stability.

## 11.4 Boundaries of Generalization

Of course, the generalization of this project still has boundaries:

- Current experimental subjects are mainly concentrated on `validator.js`-style functions;
- It has not been extended to larger, multi-module, or state-heavy complex system-level software;
- Therefore, it cannot directly claim the method has generalized to all software testing scenarios.

A more cautious conclusion is:

**This method has demonstrated good cross-function, cross-mode, and cross-version stability on validator-style functions in real open-source libraries, but generalization to more complex software systems still needs subsequent verification.**

# 12. Location of Various Artifacts in the Project

Centered around the implementation and experimental analysis of this project's method, the following main artifacts have been formed:

1. `Input`
   - Requirements and project code base, from `validator.js` documentation and source code.
2. `Tool artifact`
   - Prompts, model call logic, and model-generated JSON test sets, located in `agent_toolkit` related directories.
3. `Generated output`
   - Test generation results for each mode / agent, located in `agent_toolkit/outputs`.
4. `Experimental analysis`
   - Including Correctness, various Coverage metrics, Golden Tests Match Ratio, Avg Tokens cost analysis, and multiple rounds of ablation experiments, found in Sections 6 and 7 of this article and various `experiment_*.md`.
5. `Project report`
   - This document is `report_en.md`; original analysis materials are found in various experimental reports.

The model used in this project is `gpt-4o` as recorded in the experimental logs, and token usage is recorded to support cost analysis.

# 13. Current Status Analysis and Future Evolution

## 13.1 Attribution of Five Failure Mechanisms

Based on an in-depth analysis of failed test cases, this project summarizes five typical failure mechanisms of LLMs in test generation, providing a clear direction for future targeted optimization:

1. **Assertion Direction Error**: The model has a biased understanding of semantics, misjudging valid inputs as invalid (or vice versa), reflecting limitations in fundamental logical reasoning.
2. **Parameter Structure Error**: Generated parameters do not conform to interface specifications, data types, or option combinations, due to insufficient extraction of API constraints.
3. **Semantic Drift**: Particularly in white-box mode, over-reliance on code path logic without being constrained by requirements leads to generated tests deviating from real business needs.
4. **Boundary Detail Instability**: Shows instability when handling boundary cases such as extreme values, special characters, and long strings, reflecting fluctuations in the model's precise numerical reasoning.
5. **Assembly and Merging Conflict**: During multi-iteration patch merging, redundant tests might be introduced, or the consistency of the original test suite might be compromised.

## 13.2 Comprehensive Cost-Benefit Model

Experiments indicate that Token consumption alone is insufficient to measure the engineering feasibility of the tool. We recommend a more comprehensive cost criterion in future evaluations:
**Comprehensive Cost = Model Token Cost + End-to-End Generation Latency + Human Review and Repair Cost**
While the `improved agent` significantly boosts coverage, it also entails higher Token expenditures and interaction latency. If the Correctness metric does not improve synchronously, the increased burden of human review could offset the efficiency gains brought by automation.

## 13.3 Evolution from Monolithic Models to "Multi-Role Collaboration"

The current system primarily relies on the iteration path of a Single Agent. The future direction is toward a Multi-Role Collaboration (Multi-Role Chat) architecture:

- **Domain Expert**: Responsible for deep decomposition of requirements and business logic.
- **Test Coder**: Responsible for specific test code implementation.
- **Quality Reviewer**: Responsible for identifying assertion loopholes and providing negative feedback.
  This "Cross-Review" mechanism is expected to maintain the bottom line of correctness through internal feedback loops while chasing maximal coverage.

### 13.4 Expansion of Validation Scope

The project has currently been validated on stateless pure function libraries like `validator.js`, where input-output logic is straightforward. Future research needs to expand this method to real industrial systems involving global state, asynchronous calls, and file system/database persistence with complex state transitions (Stateful) to fully evaluate the robustness of the method.

## 14. Project Division of Labor

The project group consists of 4 members, with the division of roles as follows:

| Member | Role | Key Responsibilities |
|---|---|---|
| **黄一和 (Project Leader)** | Overall Design & Reporting | Responsible for project design, experimental scheme design, analysis of results, technical report writing, and project Presentation. |
| **夏浩博** | Core Algorithm Implementation | Responsible for the engineering implementation and prompt tuning of the core algorithms (iterative logic of the `improved agent`). |
| **关镜文** | Experimental Pipeline | Responsible for the implementation of the automated experimental pipeline based on `validator.js`, including Coverage collection and data interface development. |
| **黄弋涵** | Results & Presentation Layout | Responsible for the organization and summarization of experimental results, data statistics, and the design and production of the defense PPT. |

## 15. Conclusion

This project implemented and evaluated an LLM-driven test generation tool on the real open-source function library `validator.js`, centering on systematic design and verification of the `improved agent`.

The following conclusions can finally be drawn:

**1. Through stronger prompt design and white-box iteration mechanisms, the `improved agent` can, on real open-source projects, generate test cases with significantly higher Coverage and Golden Tests Match Ratio by sacrificing 0-10% of Correctness and increasing the cost by 1-2 times.**

**2. AI is best suited as a test design accelerator and Coverage exploration tool; however, high-reliability test quality still requires more robust constraint mechanisms and human participation in correction.**

## Appendix A: Main Prompts Used in This Project

The following shows the most core prompt templates used in this project. For ease of reading, placeholders are used to represent dynamically inserted content.

## A.1 `naive` System Prompt

```
Generate a conservative first-pass validator.js-style test suite.
Prefer obvious behaviors, obvious edge cases, and small grouped coverage over
broad exploration.
Keep the suite intentionally naive: do not optimize for completeness, novelty, or
deep branch hunting.

Always return valid JSON only.
JSON schema:
{
  "obligations": [],
  "test_groups": [
    {
      "title": "string",
      "validator": "target validator name",
      "args": [ { } ],
      "valid": ["..."],
      "invalid": ["..."],
      "rationale": "string",
      "obligations": ["OB-1", "OB-2"]
    }
  ]
}

Requirements:
- Group tests by option profile.
- obligations may be an empty list.
- Prefer a small number of clear groups over many speculative groups.
- Prefer common cases and easy-to-justify boundaries.
- Avoid complex combinations unless they are directly implied by the requirement.
- Never repeat the same JSON key within one object.
- Keep the response concise enough to remain valid JSON.
- Do not output markdown.
```

## A.2 `naive blackbox` Prompt

```
Task: generate black-box tests for {validator_name}.

Read the requirement and directly produce a small first-pass grouped
validator.js-style test suite.
Stay conservative and only cover behaviors that are explicit or strongly implied
by the requirement.
Do not spend effort extracting formal obligations.
Use an empty obligations list unless there is a very obvious single high-level
rule to note.

Black-box guidance:
- prefer common positive and negative examples
- use only a few clearly differentiated groups
- avoid speculative option interactions
- stop at a modest baseline instead of trying to be comprehensive

Requirement specification:
```

```
{requirement_spec}
```

## A.3 `naive whitebox` Prompt

```
Task: generate white-box tests for {validator_name}.

Read the requirement and source code, then directly produce a small first-pass
grouped validator.js-style test suite.
Use the source code only to notice obvious branches or guards, not to
aggressively chase coverage.
Do not try to build a detailed obligation hierarchy.
Use an empty obligations list unless there is a very obvious single high-level
rule to note.

Requirement specification:
{requirement_spec}

Source code:
{source_code}

Output guidance:
- keep the suite modest in size
- each group should focus on one option profile or one branch family
- prefer obvious branches and obvious edge cases over exhaustive exploration
- avoid patch-like additions or coverage-maximizing behavior
- keep the result as a conservative baseline, not an ambitious suite
```

## A.4 `naive whitebox_code_only` Prompt

```
Task: generate white-box tests for {validator_name} using source code only.

Read the source code and directly produce a small first-pass grouped
validator.js-style test suite.
There is no external requirement specification in this mode.
Infer externally visible behavior conservatively from obvious guards, branches,
and option handling in the code.
Do not try to build a detailed obligation hierarchy.
Use an empty obligations list unless there is a very obvious single high-level
rule to note.

Source code:
{source_code}

Output guidance:
- keep the suite modest in size
- each group should focus on one option profile or one obvious branch family
- prefer obvious branches, guards, and obvious edge cases over exhaustive
exploration
- avoid speculative behavioral claims that are not well supported by the code
- keep the result as a conservative baseline, not an ambitious suite
```

## A.5 `improved` System Prompt

```
You are an expert software testing agent.
You generate high-quality black-box and white-box tests for validator-style
functions.
You must reason conservatively, preserve correctness, and improve coverage with
small precise patches.

Always return valid JSON only.
JSON schema:
{
  "obligations": [
    {
      "id": "string",
      "kind": "blackbox|whitebox",
      "rule": "string",
      "why": "string",
      "minimal_valid": "string or null",
      "minimal_invalid": "string or null"
    }
  ],
  "test_groups": [
    {
      "title": "string",
      "validator": "target validator name",
      "args": [ { } ],
      "valid": ["..."],
      "invalid": ["..."],
      "rationale": "string",
      "obligations": ["OB-1", "OB-2"]
    }
  ]
}

Requirements:
- Group tests by option profile.
- For broad black-box APIs with many documented options, expand into multiple
semantically distinct groups instead of collapsing everything into one default
group.
- Keep each group narrow: a group should usually cover one baseline partition
family or one closely related option-sensitive behavior.
- Prefer more distinct categories when the requirement clearly documents many
switches, constraints, or acceptance/rejection modes.
- Keep patches minimal and avoid redundant duplicates.
- Prefer minimally different cases that flip one rule at a time.
- Never repeat the same JSON key within one object.
- Keep the response concise enough to remain valid JSON.
- Do not output markdown.
```

## A.6 `improved blackbox` Prompt

```
Task: generate black-box tests for {validator_name}.

Apply standard black-box testing methods aggressively:
- equivalence partitioning
- boundary value analysis
- option-combination testing
- negative-input testing
- special-case and rare-option exploration

Step 1: extract explicit black-box obligations from the requirement.
Step 2: cluster the obligations into semantically distinct black-box categories.
Step 3: expand each category into grouped validator.js-style tests.
Step 4: self-check whether rare options, boundary-adjacent cases, invalid
partitions, and documented option flips were skipped.

Requirement specification:
{requirement_spec}

Output guidance:
- first identify the major requirement-visible categories before writing any
examples
- keep one coherent option profile or one closely related rule family per group
- do not collapse many unrelated options into one catch-all "default" group
- if the requirement documents many independent switches or acceptance/rejection
rules, reflect them in multiple groups
- cover both baseline behavior and option-sensitive deviations
- prefer broad category exploration, but keep each example requirement-grounded
and likely correct
- for every option-sensitive group, choose examples whose validity actually
changes because of that option profile
- if a documented option effect is only observable together with another option
value, configure both so the distinction is real
- do not claim an example is invalid for an option if the requirement text still
permits it under the provided args
- prefer paired examples that differ by one rule or one option flip when
demonstrating requirement changes
- when a default already allows a behavior, do not present enabling the same
behavior as a meaningful new category unless another argument makes the contrast
observable
- when uncertain, choose fewer examples inside a group rather than merging
unrelated categories
- include multiple concrete valid and invalid examples only when they cover
meaningfully different cases
- try to ensure most obligations are represented by at least one dedicated or
clearly focused group

Self-audit before finalizing:
- Did you create separate groups for materially different option families instead
of one oversized group?
- Did you include default behavior, strictness-raising options, permissive
options, whitelist/blacklist style filters, and length-related behavior when
documented?
```

```
- For each option-focused group, would at least one valid/invalid pair change
outcome if that option were flipped back?
- Did you accidentally use examples whose expected result depends on undocumented
implementation details rather than the visible requirement?
- Are your expectations aligned with the requirement text rather than guessed
from hidden implementation details?
```

## A.7 `improved whitebox` Prompt

```
Task: generate white-box tests for {validator_name}.

Treat this as requirement-constrained white-box testing.
Use both the intended external behavior and the implementation structure.

Apply standard white-box testing methods aggressively:
- statement coverage
- branch coverage
- condition-oriented reasoning
- early-return triggering
- helper-sensitive path exploration
- requirement-grounded partition checking

Step 1: derive black-box obligations from the requirement.
Step 2: enumerate structural white-box obligations directly from the code.
Step 3: merge overlaps and contradictions.
Step 4: expand them into many grouped validator.js-style tests.
Step 5: self-check whether early returns, helper-dependent checks, option flips,
boundary-triggered branches, and requirement-visible edge cases were skipped.

Requirement specification:
{requirement_spec}

Source code:
{source_code}

Output limits:
- each group should focus on one option profile or one branch family
- explore broadly when requirement-visible behavior and source-level branches
genuinely justify it
- do not inflate obligations or groups just to reach a target count
- prefer a compact suite that still covers materially different paths and edge
cases
```

## A.8 `improved whitebox_code_only` Prompt

```
Task: generate white-box tests for {validator_name} using source code only.

Treat this as code-only white-box testing.
There is no external requirement specification in this mode.
Use the implementation structure to infer externally visible behavior
conservatively.

Apply standard white-box testing methods aggressively:
```

```
- statement coverage
- branch coverage
- condition-oriented reasoning
- early-return triggering
- helper-sensitive path exploration
- conservative behavior inference from code-visible contracts

Step 1: enumerate structural white-box obligations directly from the code.
Step 2: infer only well-supported externally visible behaviors from those
structures.
Step 3: expand them into grouped validator.js-style tests.
Step 4: self-check whether early returns, helper-dependent checks, option flips,
and boundary-triggered branches were skipped.

Source code:
{source_code}

Output limits:
- each group should focus on one option profile or one branch family
- explore broadly when the source code clearly justifies it
- do not inflate obligations or groups just to reach a target count
- prefer a compact suite that still covers materially different paths and edge
cases
- do not make speculative claims about undocumented behavior unless the code
strongly supports them
```

## A.9 `improved` White-box Iteration Prompt (Final Version)

```
You are improving an existing {mode} test suite for {validator_name}.
You are given only coverage details and fine-grained uncovered white-box
locations.
You must use those uncovered positions as exact white-box targets when updating
the suite.
You must NOT use any golden tests, repository test overlap analysis, or external
oracle hints.

Goal:
- propose a small patch that helps the current suite target more uncovered code
locations
- preserve the current suite unless a new patch is clearly justified by uncovered
locations
- keep the patch conservative and additive whenever possible
- keep the suite conservative and low-redundancy

Return strict JSON with top-level keys obligations and test_groups.
The returned test_groups must be patch groups only, not a full rewritten suite.

Black-box requirement source:
{requirement_spec_or_none}

Current full test suite from the previous iteration:
{generated_groups}

Coverage details allowed for improvement:
{evaluation_feedback}
```

```
Instructions:
- You must use only the requirement (if provided), the current suite, coverage
details, and uncovered details.
- Do not infer or use any correctness signal, pass/fail summary, failed cases, or
hidden oracle information.
- Treat uncovered_details as exact white-box targets, not as vague hints.
- Use the previous full suite as the baseline and return only the incremental
patch groups.
- Do not rewrite, delete, or duplicate the existing groups already shown in the
current suite.
- Add only groups that cover meaningfully new targets beyond the current suite.
- Prefer at most 3 patch groups, each focused on one concrete uncovered condition
or path family.
- If an uncovered branch or statement cannot be operationalized from the provided
coverage details, do not guess.
- If uncovered_details are provided, use them only as white-box targets.
- If uncovered_details are not provided, do not infer any hidden coverage gap.
- Prefer minimal option-profile-specific groups.
- Avoid broad extrapolation; every meaningful update should correspond to a
concrete uncovered position or condition.
- Do not mention or infer any golden test information.
```

## A.10 JSON Repair Prompt

```
Repair the following invalid JSON into strict valid JSON.
Keep the same schema with top-level keys obligations and test_groups.
Remove duplicated keys, remove obviously repeated corrupted fragments, and
preserve as much valid content as possible.
Return JSON only.

{raw_response}
```

## A.11 Group Completion Prompt

```
The previous {mode} generation for {validator_name} produced obligations but no
usable test_groups.
Using the obligations below, generate non-empty validator.js-style test_groups
now.
Return strict JSON with top-level keys obligations and test_groups.
Keep the obligations unchanged unless a tiny correction is necessary.
test_groups must be non-empty.
Each group must contain validator, args, valid, invalid, title, rationale,
obligations.

Original task:
{original_prompt}

Existing obligations:
{obligations}
```